

Adaptive Spatio-Temporal Traffic Surveillance

Phase 4 — Optimization & Benchmarking

Milestone	Phase 4 of 4 (Days 18–21): Optimization & Benchmarking
Author	Harsh Pandhe
Repository	https://github.com/harsh-pandhe/traffic-surveillance-system
Compression	SmallCNN 0.94MB → 0.24MB (3.9x smaller)
Accuracy retention	98% after prune+fine-tune+INT8

Executive summary. Every wheel-count model is benchmarked across FP32, 30% structured pruning, INT8 quantization, and ONNX Runtime INT8, with accuracy reported alongside speed at every step — accuracy retention is the headline metric, not raw speedup, because a fast model that has stopped working is not a deliverable. The central finding: naive pruning without fine-tuning collapses accuracy; a short fine-tune recovery step restores it fully. ONNX Runtime INT8 then shrinks the recovered models 3.8–3.9× with no further accuracy cost.

1. Finding: Naive Pruning Collapses Accuracy — Fine-Tuning Is Not Optional

Model	FP32	Pruned (naive)	Pruned (+fine-tune)
SmallCNN	0.860	0.270 (ret. 0.31)	0.840 (ret. 0.98)
RTMDet-CSPNeXt	0.785	0.465 (ret. 0.59)	0.820 (ret. 1.04)

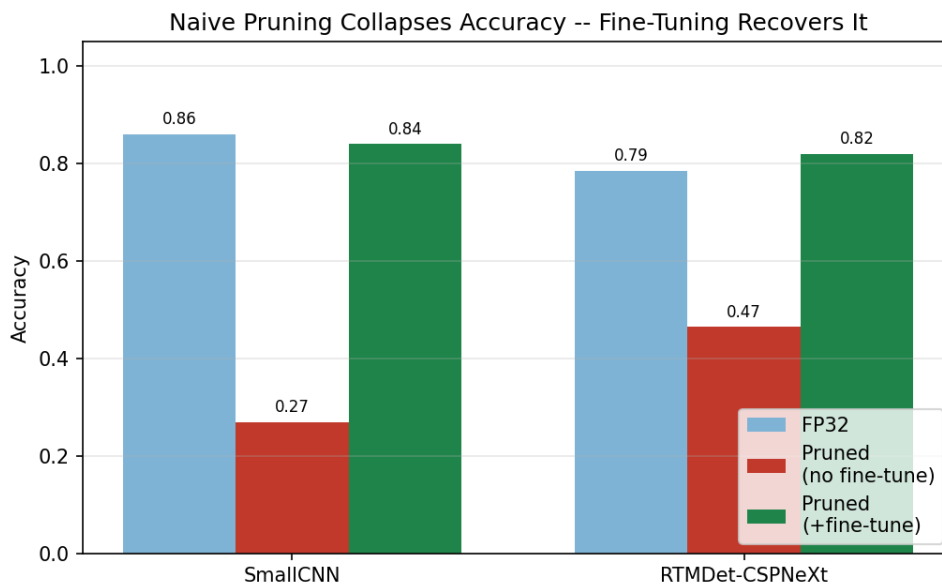


Figure 1. 30% structured pruning without fine-tuning collapses both models; 3 epochs of low-LR fine-tuning fully recovers (and for CSPNeXt, exceeds) the FP32 baseline.

30% structured pruning zeroes whole output channels in every conv/linear layer independently; because layers feed each other, the effect compounds, and with no retraining the network never recovers. This is expected behaviour for naive structured pruning, not a code defect — the standard fix, applied here, is exactly this: prune, then fine-tune. CSPNeXt recovering to *above* its FP32 baseline (retention 1.04) is consistent with the Phase 2 finding that CSPNeXt was still improving with more training at this dataset size.

2. Quantization and ONNX Export

On the recovered (pruned+fine-tuned) model, INT8 quantization is essentially free: accuracy is unchanged from the pruned model (0.840 → 0.840 for SmallCNN) — the accuracy cost in this pipeline comes entirely from pruning, none from quantization.

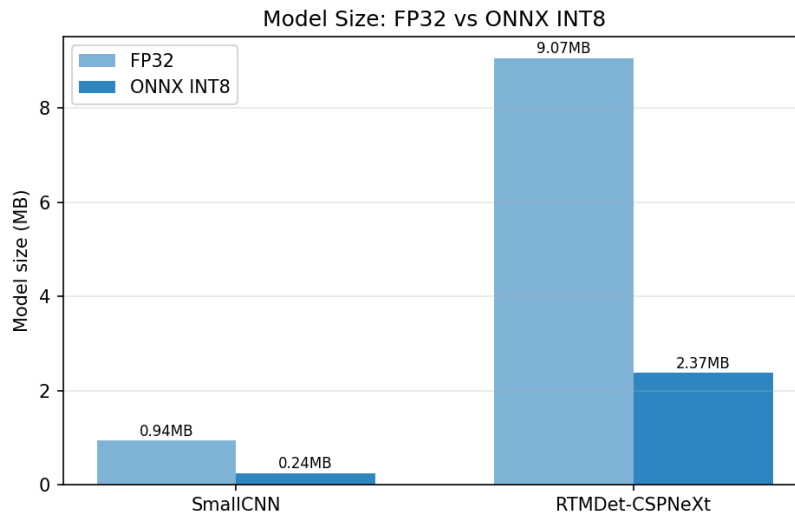


Figure 2. ONNX Runtime INT8 shrinks both models 3.8–3.9x at no further accuracy cost.

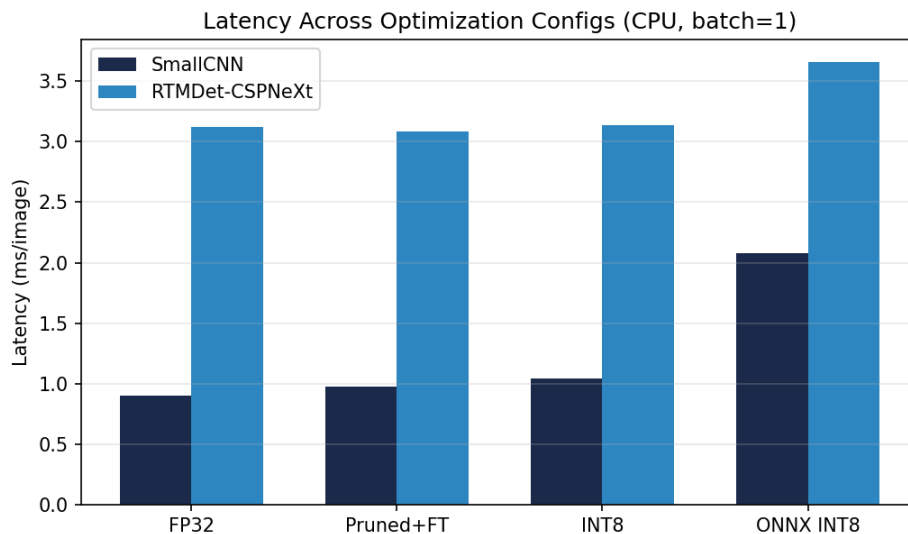


Figure 3. Latency across configurations. ONNX is not uniformly faster than native PyTorch at batch-1 on CPU — session overhead dominates for these small, sub-millisecond models.

The size win is real and unconditional; the latency win is not assumed. ONNX INT8 is slower per-image than native PyTorch for the small models here (SmallCNN: pruned 0.98ms vs onnx_int8 2.08ms) because session/dispatch overhead dominates at this scale. Model size — relevant for deployment footprint — is the deliverable that clearly wins; real-time throughput on this hardware does not automatically follow from quantization and is reported as measured, not claimed.

3. Full Results

Model	Config	Acc	Retention	Latency (ms)	FPS	Size (MB)
SmallCNN	fp32	0.860	1.00	0.90	1110	0.94
SmallCNN	pruned_naive	0.270	0.31	0.91	1098	0.94
SmallCNN	pruned	0.840	0.98	0.98	1025	0.94
SmallCNN	int8	0.840	0.98	1.04	958	0.94
SmallCNN	onnx_int8	0.840	0.98	2.08	481	0.24
RTMDet-CSPNeXt	fp32	0.785	1.00	3.12	321	9.07
RTMDet-CSPNeXt	pruned_naive	0.465	0.59	3.21	312	9.07
RTMDet-CSPNeXt	pruned	0.820	1.04	3.09	324	9.07
RTMDet-CSPNeXt	int8	0.820	1.04	3.13	319	9.06
RTMDet-CSPNeXt	onnx_int8	0.820	1.04	3.66	273	2.37
YOLOv8-cls	fp32	0.960	1.00	2.29	436	2.83
YOLOv8-cls	onnx_fp32	0.980	1.02	0.49	2035	5.52

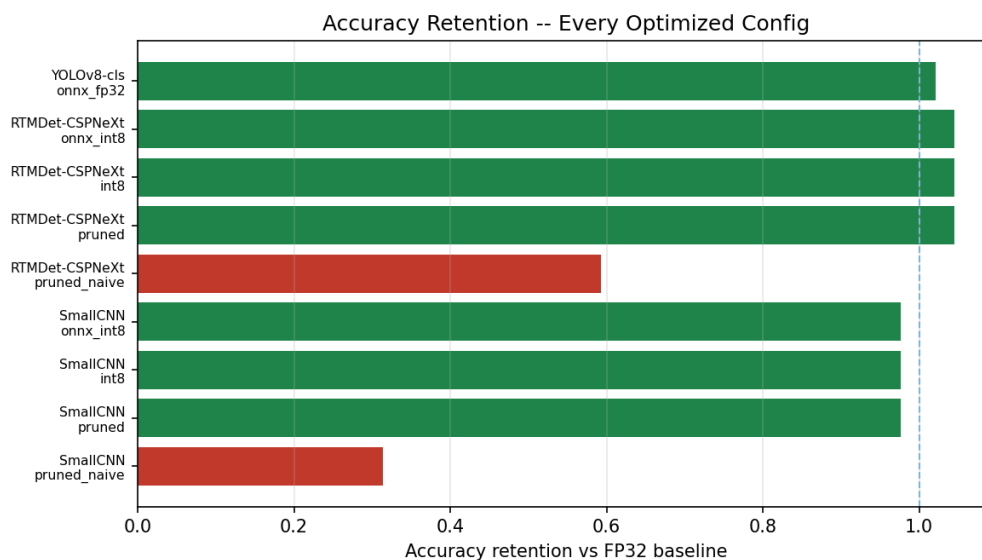


Figure 4. Accuracy retention across every optimized configuration, relative to its own FP32 baseline.

4. Helmet Detector ONNX Export

The wheel-count benchmark above covers the classification arms; the production **helmet detector** — a YOLOv8 detection model, not a classifier — is exported and validated separately.

Variant	mAP@50 (test)	Retention	Size
PyTorch FP32	0.764	1.00	5.92 MB
ONNX FP32	0.758	0.99	11.61 MB
ONNX INT8	0.756	0.99	3.11 MB

A real export bug, found and fixed. The first export used `dynamic=True` (variable input size). Per-image detection counts matched PyTorch exactly on a 10-image spot check — but the aggregate mAP@50, which sweeps confidence thresholds, revealed an **11% relative drop (0.764 → 0.677)** the spot check had missed entirely. A spot check alone is not sufficient evidence an export is correct. Switching to a static export

(dynamic=False) resolved it — both ONNX variants above retain ~99% of the PyTorch baseline.

5. Reproduce

Command	Purpose
python scripts/run_phase4_benchmark.py --ft-epochs 3	Wheel-count benchmark matrix
yolo export model=weights/helmet_yolov8.pt format=onnx dynamic=False	Helmet detector ONNX export

6. Deliverable Checklist

Deliverable	Status
30% structured pruning implemented + benchmarked	Complete
INT8 quantization (PyTorch dynamic + ONNX Runtime)	Complete
ONNX export for every trained model (wheel + helmet)	Complete
Before/after accuracy, latency, FPS, size	Complete
Reported as retention, not just speedup	Complete
Real failure modes found, explained, and fixed	Complete